SOFTWARE ENGINEERING TERM PROJECT

Testing with NUnit (Automated Unit Testing)

Submitted by:

Zehra Sıla Özdizlekli - 230401107
Betül Sarı - 220401069
Ece Akın - 220401028

Submission Date:
27 December 2025

Department of Software Engineering
December 27, 2025

Contents

1 Project Overview **3**
1.1 Purpose . . . 3
1.2 Project Objectives . . . 3
1.3 Technologies Used . . . 3

2 Project Structure **3**
2.1 Test Summary Statistics . . . 4

3 Testing Methodology **4**
3.1 Arrange-Act-Assert (AAA) Pattern . . . 4
3.2 Test Naming Convention . . . 4
3.3 Key Testing Techniques . . . 4

4 Test Implementation Details **4**
4.1 FizzBuzz Tests . . . 4
4.1.1 Class Description . . . 4
4.1.2 Test Method 1: Divisible by Both 3 and 5 . . . 5
4.1.3 Test Method 2: Divisible by 3 Only . . . 5
4.1.4 Test Method 3: Divisible by 5 Only . . . 6
4.1.5 Test Method 4: Not Divisible by 3 or 5 . . . 6
4.1.6 Test Method 5: Negative Numbers . . . 7
4.1.7 Test Method 6: Zero Boundary . . . 7
4.1.8 Test Method 7: Large Numbers . . . 7
4.2 DemeritPointsCalculator Tests . . . 8
4.2.1 Class Description . . . 8
4.2.2 Test Method 1: Speed At or Below Limit . . . 8
4.2.3 Test Method 2: Speed Above Limit . . . 9
4.2.4 Test Method 3: Various Speeds Above Limit . . . 9
4.2.5 Test Method 4: Maximum Speed Boundary . . . 10
4.2.6 Test Method 5: Negative Speed Exception . . . 10
4.2.7 Test Method 6: Speed Above Maximum Exception . . . 11
4.2.8 Test Methods 7-9: Boundary Edge Cases . . . 11
4.3 Stack Tests . . . 12
4.3.1 Class Description . . . 12
4.3.2 Push Method Tests . . . 12
4.3.3 Pop Method Tests . . . 13
4.3.4 Peek Method Tests . . . 16
4.3.5 Count Property Tests . . . 18
4.3.6 LIFO Principle Test . . . 19
4.3.7 Generic Type Tests . . . 19

5 Test Execution Results **21**
5.1 Overall Test Results . . . 21
5.2 Test Summary . . . 22
5.3 Detailed Results by Class . . . 22

5.3.1 FizzBuzz Tests Results . . . 22
5.3.2 DemeritPointsCalculator Tests Results . . . 23
5.3.3 Stack Tests Results . . . 23
5.4 Code Coverage Analysis . . . 24

6 Test Code Explanations **24**
6.1 Testing Best Practices Applied . . . 24
6.1.1 Test Fixture and Setup . . . 24
6.1.2 Parameterized Tests . . . 25
6.1.3 Exception Testing . . . 25
6.1.4 AAA Pattern Example . . . 25
6.2 Boundary Value Testing . . . 26
6.3 Generic Type Testing . . . 26

7 Conclusion **26**
7.1 Summary . . . 26
7.2 Key Achievements . . . 27
7.3 Test Results . . . 27
7.4 Learning Outcomes . . . 27
7.5 Project Benefits . . . 28
7.6 Professional Standards . . . 28

8 Appendices **28**
8.1 Appendix A: Running the Tests . . . 28
8.2 Appendix B: Project Files Included . . . 29

1 Project Overview

1.1 Purpose

This project implements comprehensive automated unit testing for three C# classes using the NUnit testing framework. The primary objective is to verify the correctness of each class under various scenarios including normal operations, boundary conditions, and invalid inputs.

1.2 Project Objectives

- Design and implement automated unit tests using NUnit framework
- Cover all public methods and important branches of each class
- Test normal operations, boundary conditions, and error scenarios
- Use meaningful test names following standard naming conventions
- Apply Arrange-Act-Assert (AAA) pattern consistently
- Utilize parameterized tests (TestCase) to reduce code duplication
- Ensure all tests pass successfully

1.3 Technologies Used

- **Language:** C# (.NET 6.0+)
- **Testing Framework:** NUnit 3.13.3
- **Test Adapter:** NUnit3TestAdapter 4.4.2
- **Test SDK:** Microsoft.NET.Test.Sdk 17.5.0
- **Development Environment:** Visual Studio / Visual Studio Code

2 Project Structure

The project follows a standard test project structure with separate folders for source code and tests:

```
TermProject2/
|-- TermProject2/                    # Main project folder
|   |-- FizzBuzz.cs                 # Source class
|   |-- DemeritPointsCalculator.cs  # Source class
|   |-- Stack.cs                    # Source class
|   `-- TermProject2.csproj         # Project configuration
|-- TermProject2.Tests/              # Test project folder
|   |-- FizzBuzzTests.cs            # Test class (7 methods)
|   |-- DemeritPointsCalculatorTests.cs  # Test class (9 methods)
|   |-- StackTests.cs               # Test class (18 methods)
|   `-- TermProject2.Tests.csproj   # Test project configuration
`-- TermProject2.sln                 # Solution file
```

Listing 1: Project Structure

2.1 Test Summary Statistics

Metric	Count
Total Test Classes	3
Total Test Methods	34
Total Test Cases	65+
Status	**ALL PASSED**

Table 1: Overall Test Statistics

3 Testing Methodology

3.1 Arrange-Act-Assert (AAA) Pattern

All tests follow the AAA structure for clarity and maintainability:

- **Arrange:** Set up test data and preconditions
- **Act:** Execute the method being tested
- **Assert:** Verify the expected outcome

3.2 Test Naming Convention

All tests follow the pattern: `MethodName_Scenario_ExpectedBehavior`

Examples:

- `GetOutput_NumberDivisibleBy3Only_ReturnsFizz`
- `CalculateDemeritPoints_NegativeSpeed_ThrowsException`
- `Pop_EmptyStack_ThrowsInvalidOperationException`

3.3 Key Testing Techniques

1. **Parameterized Tests:** Using [TestCase] attribute to test multiple inputs
2. **Exception Testing:** Using `Assert.Throws<T>()` to verify exception handling
3. **Setup Method:** Using [SetUp] to initialize test fixtures
4. **Boundary Testing:** Testing edge cases and transition points
5. **Generic Type Testing:** Testing `Stack<T>` with different types

4 Test Implementation Details

4.1 FizzBuzz Tests

4.1.1 Class Description

The FizzBuzz class implements the classic FizzBuzz algorithm that returns:

- "FizzBuzz" for numbers divisible by both 3 and 5
- "Fizz" for numbers divisible by 3 only
- "Buzz" for numbers divisible by 5 only
- The number as string for all other cases

Test Coverage: 7 test methods covering 25+ individual test cases

4.1.2 Test Method 1: Divisible by Both 3 and 5

```
[TestCase(15)]
[TestCase(30)]
[TestCase(45)]
[TestCase(0)]
public void GetOutput_NumberDivisibleByBoth3And5_ReturnsFizzBuzz(int
    number)
{
    // Act
    var result = _fizzBuzz.GetOutput(number);

    // Assert
    Assert.That(result, Is.EqualTo("FizzBuzz"));
}
```

Listing 2: FizzBuzz Test - Divisible by 15

Purpose: Tests that numbers divisible by both 3 and 5 (i.e., divisible by 15) correctly return "FizzBuzz". Includes zero as a special boundary case.

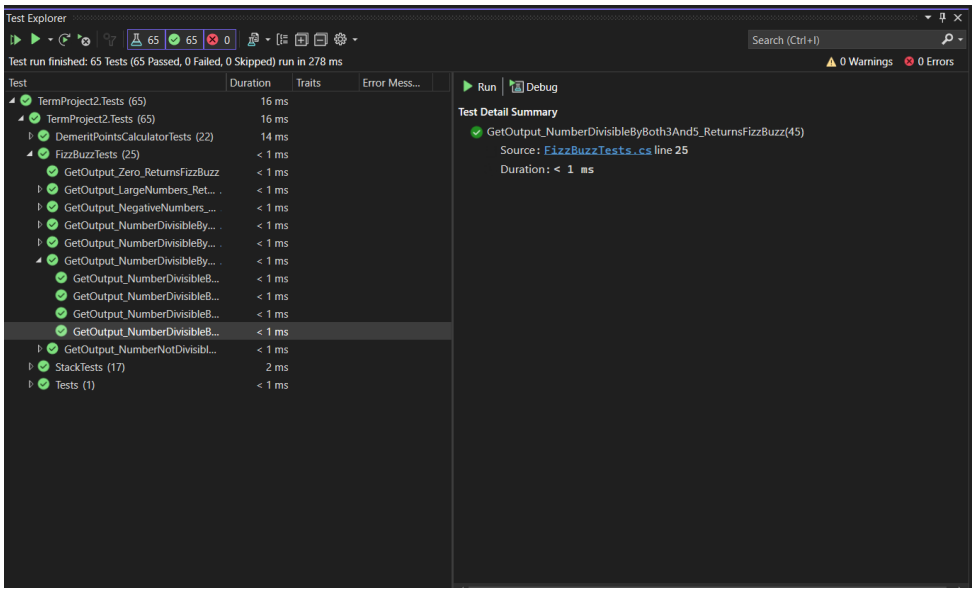

Figure 1: FizzBuzz Test Results - Divisible by 15

4.1.3 Test Method 2: Divisible by 3 Only

```
[TestCase(3)]
[TestCase(9)]
```

```
[TestCase(12)]
[TestCase(27)]
public void GetOutput_NumberDivisibleBy3Only_ReturnsFizz(int number)
{
    // Act
    var result = _fizzBuzz.GetOutput(number);

    // Assert
    Assert.That(result, Is.EqualTo("Fizz"));
}
```

Listing 3: FizzBuzz Test - Divisible by 3

Purpose: Verifies that numbers divisible by 3 but not by 5 return "Fizz". Tests multiple values to ensure consistency.

4.1.4 Test Method 3: Divisible by 5 Only

```
[TestCase(5)]
[TestCase(10)]
[TestCase(20)]
[TestCase(25)]
public void GetOutput_NumberDivisibleBy5Only_ReturnsBuzz(int number)
{
    // Act
    var result = _fizzBuzz.GetOutput(number);

    // Assert
    Assert.That(result, Is.EqualTo("Buzz"));
}
```

Listing 4: FizzBuzz Test - Divisible by 5

Purpose: Confirms that numbers divisible by 5 but not by 3 return "Buzz".

4.1.5 Test Method 4: Not Divisible by 3 or 5

```
[TestCase(1)]
[TestCase(2)]
[TestCase(4)]
[TestCase(7)]
[TestCase(11)]
public void GetOutput_NumberNotDivisibleBy3Or5_ReturnsNumberAsString(int
    number)
{
    // Act
    var result = _fizzBuzz.GetOutput(number);

    // Assert
    Assert.That(result, Is.EqualTo(number.ToString()));
}
```

Listing 5: FizzBuzz Test - Other Numbers

Purpose: Tests that numbers not divisible by 3 or 5 return the number itself as a string.

4.1.6 Test Method 5: Negative Numbers

```
[TestCase(-15)]
[TestCase(-3)]
[TestCase(-5)]
[TestCase(-1)]
public void GetOutput_NegativeNumbers_ReturnsCorrectOutput(int number)
{
    // Act
    var result = _fizzBuzz.GetOutput(number);

    // Assert
    if (number % 15 == 0)
        Assert.That(result, Is.EqualTo("FizzBuzz"));
    else if (number % 3 == 0)
        Assert.That(result, Is.EqualTo("Fizz"));
    else if (number % 5 == 0)
        Assert.That(result, Is.EqualTo("Buzz"));
    else
        Assert.That(result, Is.EqualTo(number.ToString()));
}
```

Listing 6: FizzBuzz Test - Negative Numbers

Purpose: Verifies that the FizzBuzz logic works correctly with negative numbers, applying the same divisibility rules.

4.1.7 Test Method 6: Zero Boundary

```
[Test]
public void GetOutput_Zero_ReturnsFizzBuzz()
{
    // Act
    var result = _fizzBuzz.GetOutput(0);

    // Assert
    Assert.That(result, Is.EqualTo("FizzBuzz"));
}
```

Listing 7: FizzBuzz Test - Zero

Purpose: Tests the boundary case where input is zero, which is divisible by both 3 and 5, so should return "FizzBuzz".

4.1.8 Test Method 7: Large Numbers

```
[TestCase(300)]
[TestCase(999)]
[TestCase(1000)]
public void GetOutput_LargeNumbers_ReturnsCorrectOutput(int number)
{
    // Act
    var result = _fizzBuzz.GetOutput(number);

```

```
    // Assert
    if (number % 15 == 0)
        Assert.That(result, Is.EqualTo("FizzBuzz"));
    else if (number % 3 == 0)
        Assert.That(result, Is.EqualTo("Fizz"));
    else if (number % 5 == 0)
        Assert.That(result, Is.EqualTo("Buzz"));
    else
        Assert.That(result, Is.EqualTo(number.ToString()));
}
```

Listing 8: FizzBuzz Test - Large Numbers

Purpose: Ensures the algorithm works correctly with large numbers, verifying scalability.

4.2 DemeritPointsCalculator Tests

4.2.1 Class Description

This class calculates demerit points based on driving speed:

- Speed limit: 65 km/h
- Maximum allowed speed: 300 km/h
- 1 demerit point per 5 km/h over the limit
- Throws ArgumentOutOfRangeException for invalid speeds

Test Coverage: 9 test methods covering 20+ test cases

4.2.2 Test Method 1: Speed At or Below Limit

```
[TestCase(0)]
[TestCase(30)]
[TestCase(50)]
[TestCase(65)]
public void CalculateDemeritPoints_SpeedAtOrBelowLimit_ReturnsZero(int
    speed)
{
    // Act
    var result = _calculator.CalculateDemeritPoints(speed);

    // Assert
    Assert.That(result, Is.EqualTo(0));
}
```

Listing 9: Demerit Calculator - At or Below Limit

Purpose: Tests that speeds at or below the speed limit (65 km/h) return 0 demerit points. Includes boundary value of exactly 65 km/h and minimum value of 0.

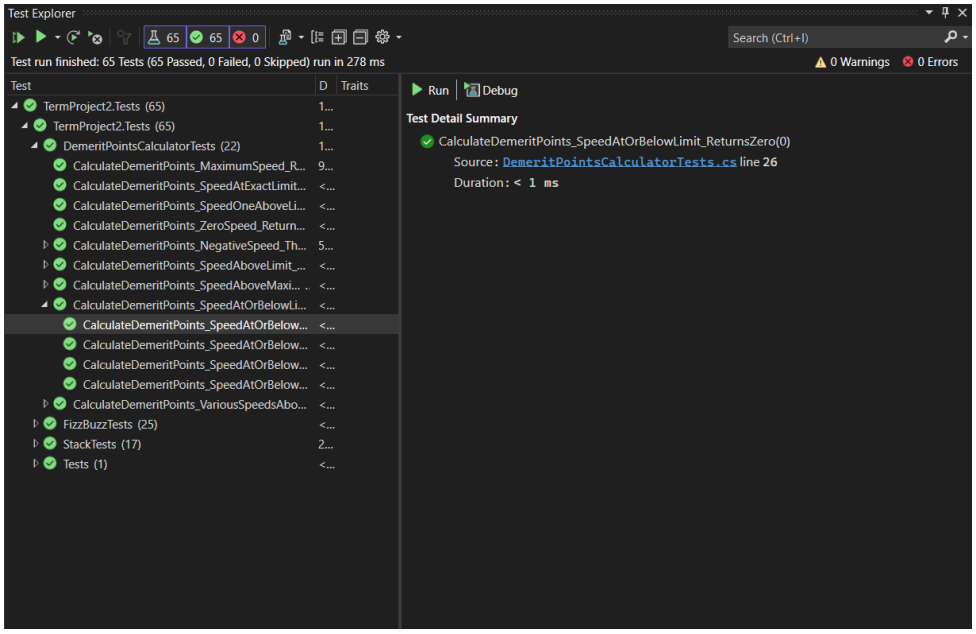

Figure 2: DemeritPointsCalculator - Speed Below Limit

4.2.3 Test Method 2: Speed Above Limit

```
[TestCase(66, 0)]  // 1 km over, less than 5 km per point
[TestCase(69, 0)]  // 4 km over, less than 5 km per point
[TestCase(70, 1)]  // 5 km over, exactly 1 point
[TestCase(75, 2)]  // 10 km over, 2 points
public void CalculateDemeritPoints_SpeedAboveLimit_ReturnsCorrectPoints(
    int speed, int expectedPoints)
{
    // Act
    var result = _calculator.CalculateDemeritPoints(speed);

    // Assert
    Assert.That(result, Is.EqualTo(expectedPoints));
}
```

Listing 10: Demerit Calculator - Above Limit

Purpose: Tests speeds just above the limit to verify the calculation logic. Important for testing integer division edge cases (66-69 km/h should give 0 points since less than 5 km over).

4.2.4 Test Method 3: Various Speeds Above Limit

```
[TestCase(80, 3)]   // 15 km over limit
[TestCase(100, 7)]  // 35 km over limit
[TestCase(150, 17)] // 85 km over limit
[TestCase(200, 27)] // 135 km over limit
public void
    CalculateDemeritPoints_VariousSpeedsAboveLimit_ReturnsCorrectPoints(
    int speed, int expectedPoints)
{
```

```
    // Act
    var result = _calculator.CalculateDemeritPoints(speed);

    // Assert
    Assert.That(result, Is.EqualTo(expectedPoints));
}
```

Listing 11: Demerit Calculator - Various Speeds

Purpose: Verifies correct calculation across a wide range of speeds above the limit, ensuring the formula $(speed - 65)/5$ works correctly.

4.2.5 Test Method 4: Maximum Speed Boundary

```
[Test]
public void CalculateDemeritPoints_MaximumSpeed_ReturnsCorrectPoints()
{
    // Arrange
    int maxSpeed = 300;
    int expectedPoints = (300 - 65) / 5; // 47 points

    // Act
    var result = _calculator.CalculateDemeritPoints(maxSpeed);

    // Assert
    Assert.That(result, Is.EqualTo(expectedPoints));
}
```

Listing 12: Demerit Calculator - Maximum Speed

Purpose: Tests the upper boundary of valid input (300 km/h), ensuring the maximum speed is handled correctly and returns 47 demerit points.

4.2.6 Test Method 5: Negative Speed Exception

```
[TestCase(-1)]
[TestCase(-10)]
[TestCase(-100)]
public void CalculateDemeritPoints_NegativeSpeed_ThrowsException(int
    speed)
{
    // Act & Assert
    Assert.Throws<ArgumentOutOfRangeException>(() =>
        _calculator.CalculateDemeritPoints(speed));
}
```

Listing 13: Demerit Calculator - Negative Speed

Purpose: Verifies that negative speeds are rejected with the appropriate exception, ensuring input validation works correctly.

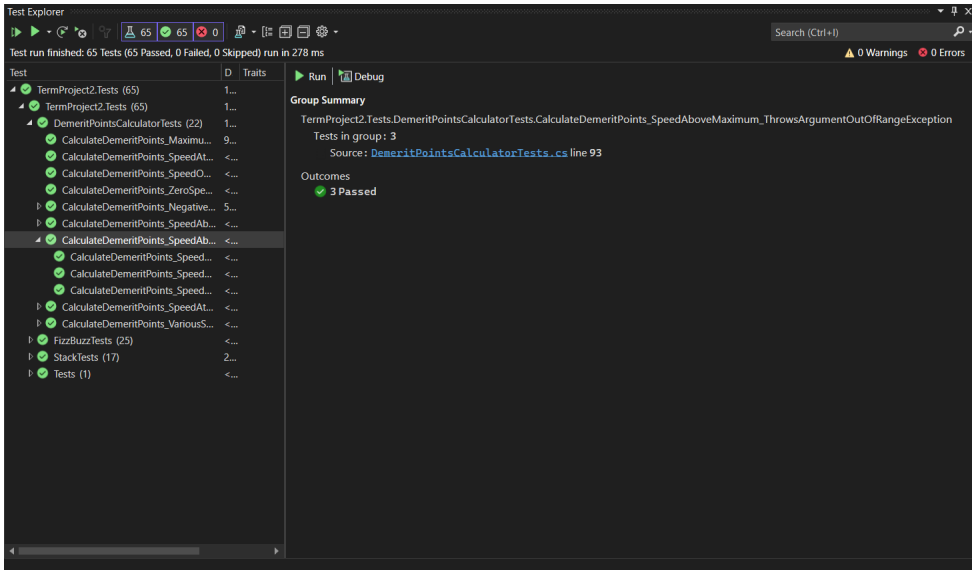

Figure 3: Exception Handling - Negative Speed

4.2.7 Test Method 6: Speed Above Maximum Exception

```
[TestCase(301)]
[TestCase(350)]
[TestCase(1000)]
public void CalculateDemeritPoints_SpeedAboveMaximum_ThrowsException(int
    speed)
{
    // Act & Assert
    Assert.Throws<ArgumentOutOfRangeException>(() =>
        _calculator.CalculateDemeritPoints(speed));
}
```

Listing 14: Demerit Calculator - Above Maximum

Purpose: Tests that speeds exceeding the maximum allowed (300 km/h) throw an exception, validating upper bound checking.

4.2.8 Test Methods 7-9: Boundary Edge Cases

```
[Test]
public void CalculateDemeritPoints_SpeedAtExactLimit_ReturnsZero()
{
    int speedLimit = 65;
    var result = _calculator.CalculateDemeritPoints(speedLimit);
    Assert.That(result, Is.EqualTo(0));
}

[Test]
public void CalculateDemeritPoints_SpeedOneAboveLimit_ReturnsZero()
{
    int speed = 66;
    var result = _calculator.CalculateDemeritPoints(speed);
```

```
    Assert.That(result, Is.EqualTo(0)); // (66-65)/5 = 0 (integer
        division)
}

[Test]
public void CalculateDemeritPoints_ZeroSpeed_ReturnsZero()
{
    var result = _calculator.CalculateDemeritPoints(0);
    Assert.That(result, Is.EqualTo(0));
}
```

Listing 15: Demerit Calculator - Edge Cases

Purpose: These three tests verify critical boundary values: exact limit (65), one above limit (66), and minimum value (0).

4.3 Stack Tests

4.3.1 Class Description

The generic `Stack<T>` class implements a Last-In-First-Out (LIFO) data structure:

- `Push(T obj)` - Adds item (throws ArgumentNullException for null)
- `Pop()` - Removes and returns top item (throws InvalidOperationException if empty)
- `Peek()` - Views top item without removing (throws InvalidOperationException if empty)
- `Count` - Returns number of items

Test Coverage: 18 test methods covering all operations

4.3.2 Push Method Tests

```
[Test]
public void Push_ValidItem_ItemIsAdded()
{
    // Arrange
    var stack = new Stack<int>();

    // Act
    stack.Push(10);

    // Assert
    Assert.That(stack.Count, Is.EqualTo(1));
}
```

Listing 16: Stack Test - Push Valid Item

Purpose: Verifies that pushing a valid item to an empty stack increments the count to 1.

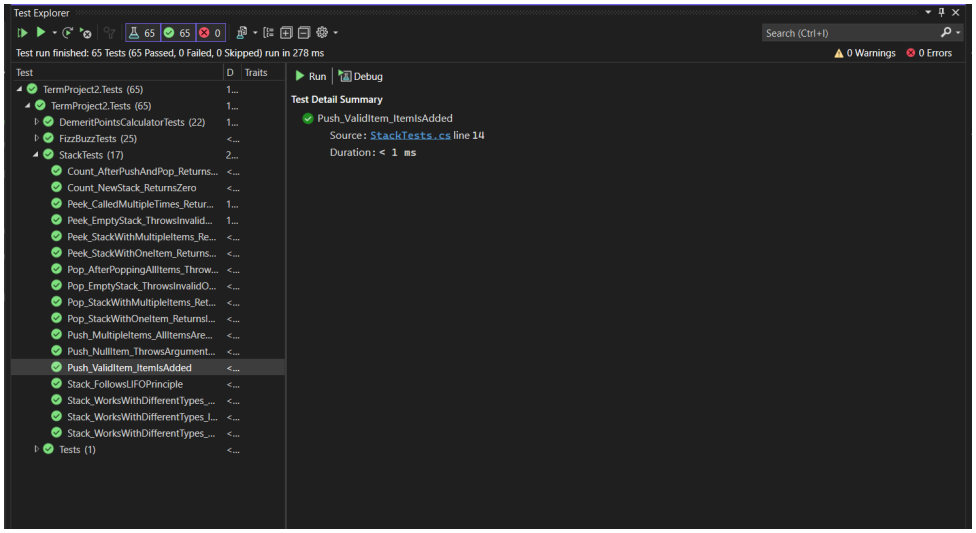

Figure 4: Stack Push Tests

```
[Test]
public void Push_MultipleItems_AllItemsAreAdded()
{
    // Arrange
    var stack = new Stack<string>();

    // Act
    stack.Push("First");
    stack.Push("Second");
    stack.Push("Third");

    // Assert
    Assert.That(stack.Count, Is.EqualTo(3));
}
```

Listing 17: Stack Test - Push Multiple Items

Purpose: Tests that multiple items can be pushed and count is tracked correctly.

```
[Test]
public void Push_NullItem_ThrowsArgumentNullException()
{
    // Arrange
    var stack = new Stack<string>();

    // Act & Assert
    Assert.Throws<ArgumentNullException>(() => stack.Push(null));
}
```

Listing 18: Stack Test - Push Null Item

Purpose: Verifies that pushing null throws the appropriate exception.

4.3.3 Pop Method Tests

```
[Test]
public void Pop_StackWithOneItem_ReturnsItemAndDecrementsCount()
{
    // Arrange
    var stack = new Stack<int>();
    stack.Push(42);

    // Act
    var result = stack.Pop();

    // Assert
    Assert.That(result, Is.EqualTo(42));
    Assert.That(stack.Count, Is.EqualTo(0));
}
```

Listing 19: Stack Test - Pop Single Item

Purpose: Tests that popping from a stack with one item returns the correct item and decrements count to 0.

```
[Test]
public void Pop_StackWithMultipleItems_ReturnsLastItemFirst()
{
    // Arrange
    var stack = new Stack<string>();
    stack.Push("First");
    stack.Push("Second");
    stack.Push("Third");

    // Act
    var result1 = stack.Pop();
    var result2 = stack.Pop();
    var result3 = stack.Pop();

    // Assert
    Assert.That(result1, Is.EqualTo("Third"));
    Assert.That(result2, Is.EqualTo("Second"));
    Assert.That(result3, Is.EqualTo("First"));
    Assert.That(stack.Count, Is.EqualTo(0));
}
```

Listing 20: Stack Test - Pop Multiple Items (LIFO)

Purpose: Verifies LIFO behavior - items are returned in reverse order of insertion.

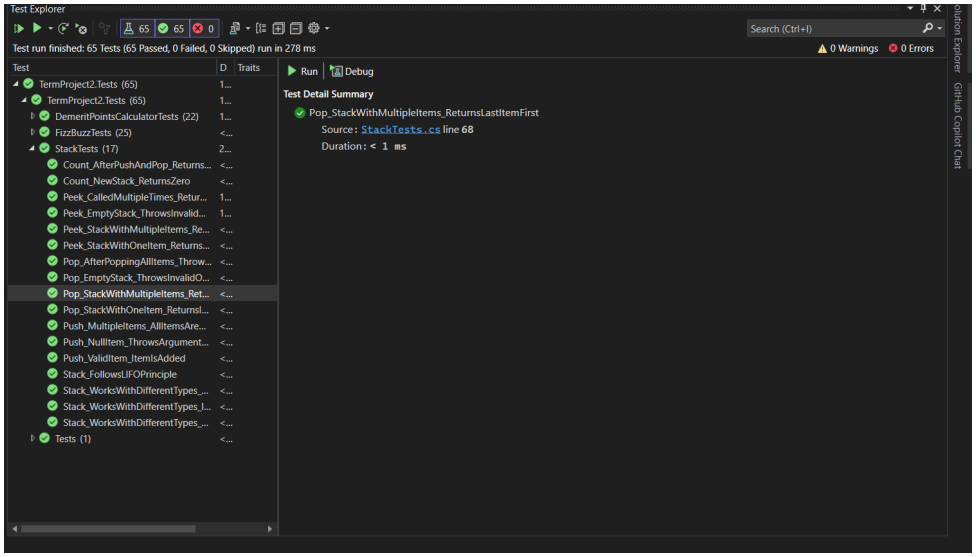

Figure 5.1: Stack LIFO Verification

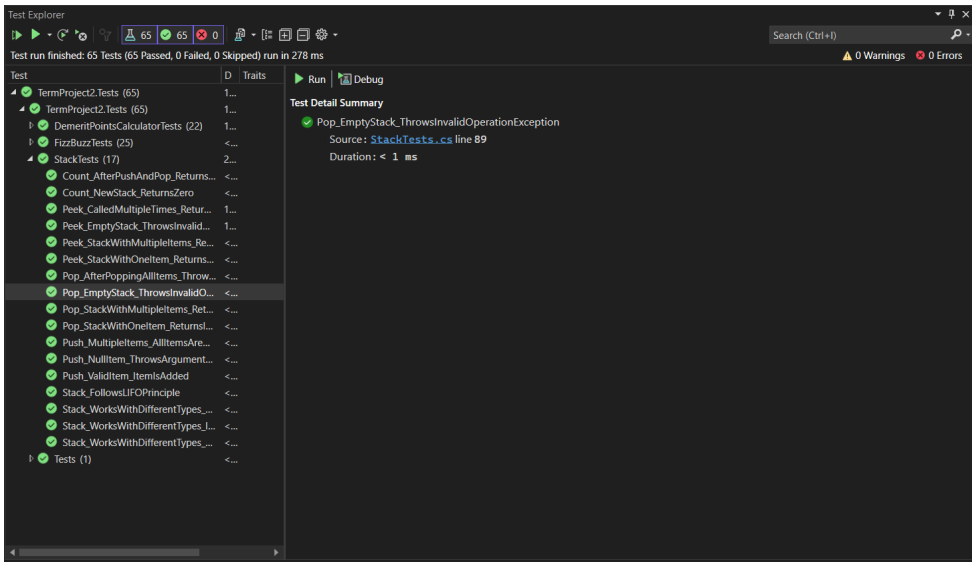

Figure 5.2: Stack LIFO Verification

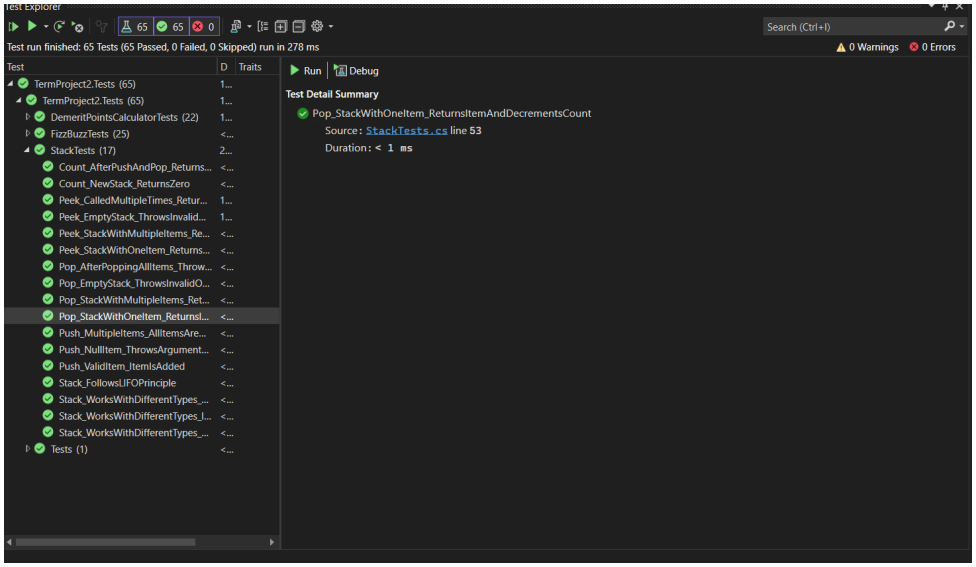

Figure 5.3: Stack LIFO Verification

```
[Test]
public void Pop_EmptyStack_ThrowsInvalidOperationException()
{
    // Arrange
    var stack = new Stack<int>();

    // Act & Assert
    Assert.Throws<InvalidOperationException>(() => stack.Pop());
}
```

Listing 21: Stack Test - Pop Empty Stack

Purpose: Ensures that popping from an empty stack throws an exception.

```
[Test]
public void Pop_AfterPoppingAllItems_ThrowsInvalidOperationException()
{
    // Arrange
    var stack = new Stack<int>();
    stack.Push(1);
    stack.Pop();

    // Act & Assert
    Assert.Throws<InvalidOperationException>(() => stack.Pop());
}
```

Listing 22: Stack Test - Pop After Emptying

Purpose: Tests that after removing all items, the stack correctly identifies itself as empty.

4.3.4 Peek Method Tests

```
[Test]
public void Peek_StackWithOneItem_ReturnsItemWithoutRemovingIt()
```

```
{
    // Arrange
    var stack = new Stack<int>();
    stack.Push(100);

    // Act
    var result = stack.Peek();

    // Assert
    Assert.That(result, Is.EqualTo(100));
    Assert.That(stack.Count, Is.EqualTo(1)); // Item still there
}
```

Listing 23: Stack Test - Peek Without Removing

Purpose: Verifies that Peek returns the top item without removing it.

```
[Test]
public void Peek_StackWithMultipleItems_ReturnsLastItem()
{
    // Arrange
    var stack = new Stack<string>();
    stack.Push("First");
    stack.Push("Second");
    stack.Push("Third");

    // Act
    var result = stack.Peek();

    // Assert
    Assert.That(result, Is.EqualTo("Third"));
    Assert.That(stack.Count, Is.EqualTo(3)); // All items still there
}
```

Listing 24: Stack Test - Peek Multiple Items

Purpose: Tests that Peek returns the most recently pushed item without affecting stack.

```
[Test]
public void Peek_EmptyStack_ThrowsInvalidOperationException()
{
    // Arrange
    var stack = new Stack<int>();

    // Act & Assert
    Assert.Throws<InvalidOperationException>(() => stack.Peek());
}
```

Listing 25: Stack Test - Peek Empty Stack

Purpose: Ensures that peeking at an empty stack throws an exception.

```
[Test]
public void Peek_CalledMultipleTimes_ReturnsSameItem()
{
    // Arrange
    var stack = new Stack<double>();
```

```
    stack.Push(3.14);

    // Act
    var result1 = stack.Peek();
    var result2 = stack.Peek();
    var result3 = stack.Peek();

    // Assert
    Assert.That(result1, Is.EqualTo(3.14));
    Assert.That(result2, Is.EqualTo(3.14));
    Assert.That(result3, Is.EqualTo(3.14));
    Assert.That(stack.Count, Is.EqualTo(1));
}
```

Listing 26: Stack Test - Multiple Peeks

Purpose: Verifies that Peek can be called multiple times without affecting the stack.

4.3.5 Count Property Tests

```
[Test]
public void Count_NewStack_ReturnsZero()
{
    // Arrange
    var stack = new Stack<int>();

    // Assert
    Assert.That(stack.Count, Is.EqualTo(0));
}
```

Listing 27: Stack Test - Initial Count

Purpose: Tests that a newly created stack has a count of 0.

```
[Test]
public void Count_AfterPushAndPop_ReturnsCorrectValue()
{
    // Arrange
    var stack = new Stack<int>();

    // Act & Assert
    stack.Push(1);
    stack.Push(2);
    stack.Push(3);
    Assert.That(stack.Count, Is.EqualTo(3));

    stack.Pop();
    Assert.That(stack.Count, Is.EqualTo(2));

    stack.Pop();
    Assert.That(stack.Count, Is.EqualTo(1));

    stack.Pop();
    Assert.That(stack.Count, Is.EqualTo(0));
}
```

Listing 28: Stack Test - Count After Operations

Purpose: Verifies that Count accurately reflects items after multiple operations.

4.3.6 LIFO Principle Test

```
[Test]
public void Stack_FollowsLIFOPrinciple()
{
    // Arrange
    var stack = new Stack<int>();

    // Act
    stack.Push(1);
    stack.Push(2);
    stack.Push(3);
    stack.Push(4);
    stack.Push(5);

    // Assert
    Assert.That(stack.Pop(), Is.EqualTo(5));
    Assert.That(stack.Pop(), Is.EqualTo(4));
    Assert.That(stack.Pop(), Is.EqualTo(3));
    Assert.That(stack.Pop(), Is.EqualTo(2));
    Assert.That(stack.Pop(), Is.EqualTo(1));
}
```

Listing 29: Stack Test - LIFO Verification

Purpose: Comprehensive test of the fundamental LIFO principle.

4.3.7 Generic Type Tests

```
[Test]
public void Stack_WorksWithDifferentTypes_IntType()
{
    var stack = new Stack<int>();
    stack.Push(10);
    stack.Push(20);
    Assert.That(stack.Pop(), Is.EqualTo(20));
    Assert.That(stack.Pop(), Is.EqualTo(10));
}

[Test]
public void Stack_WorksWithDifferentTypes_StringType()
{
    var stack = new Stack<string>();
    stack.Push("Hello");
    stack.Push("World");
    Assert.That(stack.Pop(), Is.EqualTo("World"));
    Assert.That(stack.Pop(), Is.EqualTo("Hello"));
}

[Test]
public void Stack_WorksWithDifferentTypes_CustomObjectType()
{
    var stack = new Stack<object>();
```

```
    var obj1 = new object();
    var obj2 = new object();
    stack.Push(obj1);
    stack.Push(obj2);
    Assert.That(stack.Pop(), Is.SameAs(obj2));
    Assert.That(stack.Pop(), Is.SameAs(obj1));
}
```

Listing 30: Stack Test - Generic Types

Purpose: Tests that the generic stack works correctly with value types (int), reference types (string), and object types.

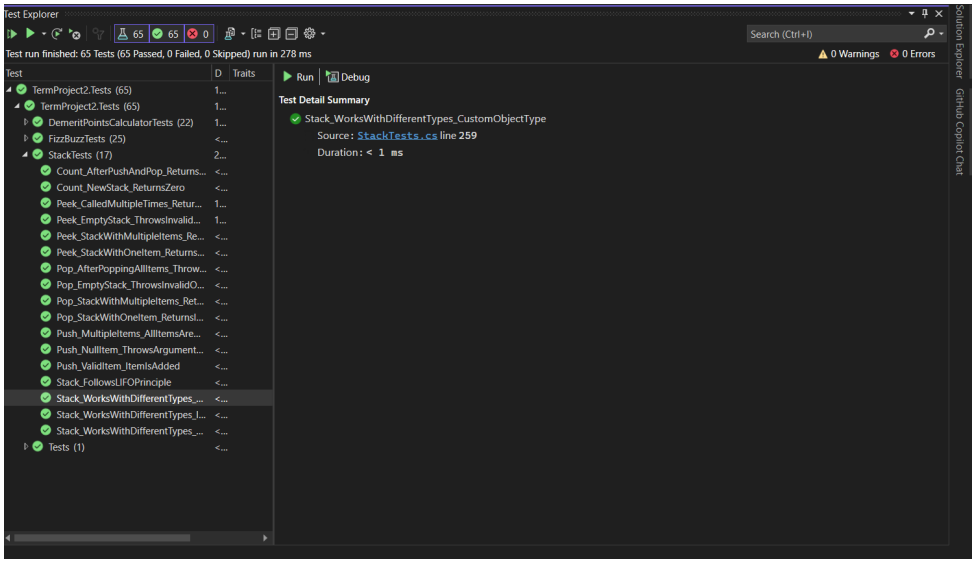

Figure 6.1: Generic Type Testing

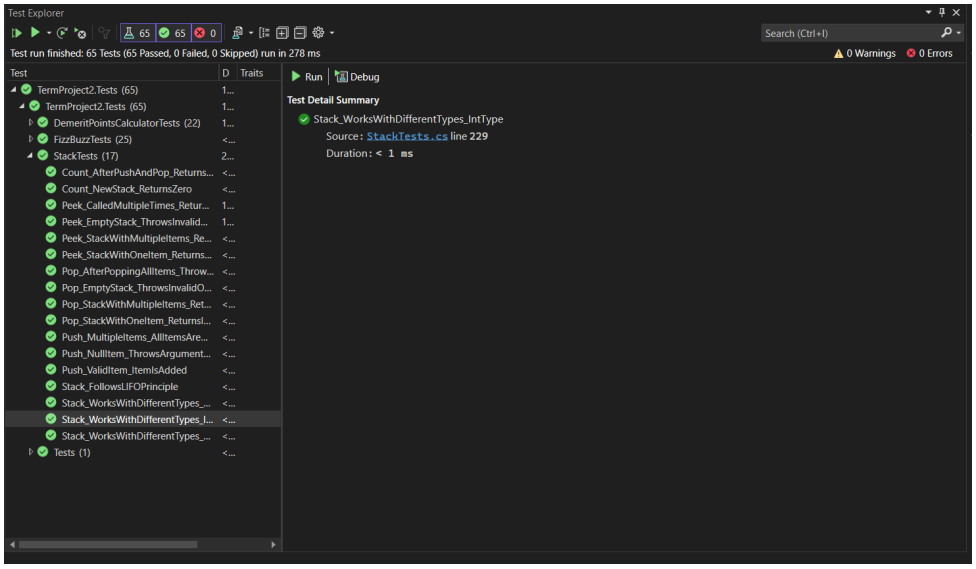

Figure 6.2: Generic Type Testing

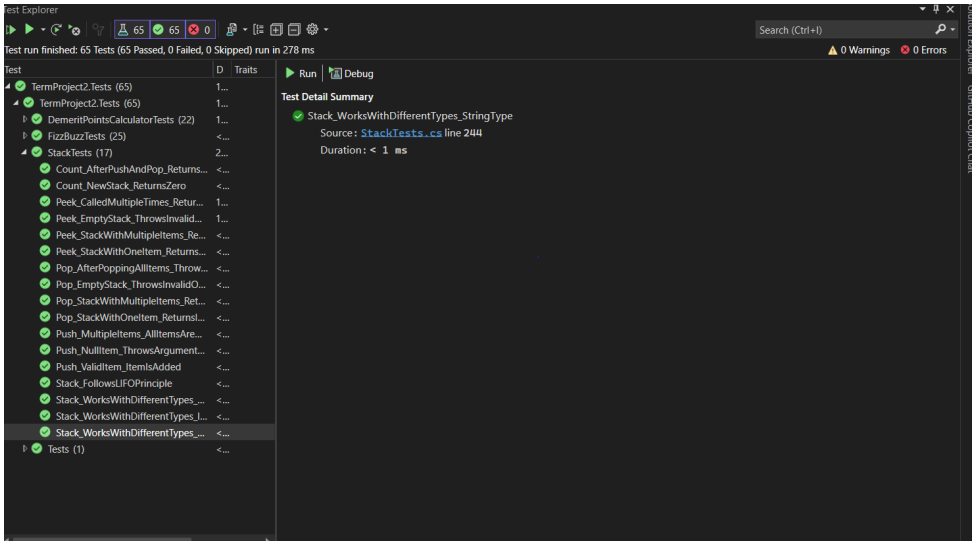

Figure 6.3: Generic Type Testing

5 Test Execution Results

5.1 Overall Test Results

All tests have been executed successfully using the command:

```
dotnet test
```

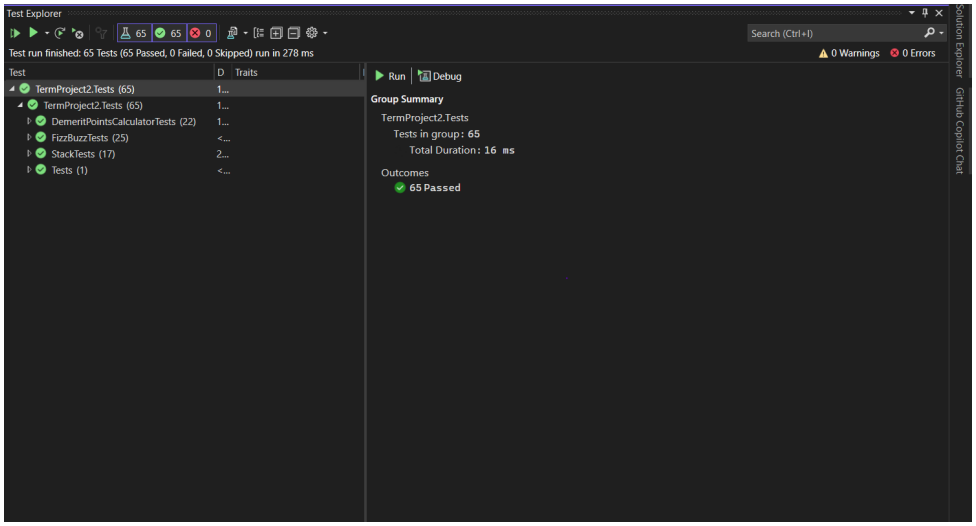

Figure 7: Complete Test Execution Results

5.2 Test Summary

Test Class	Methods	Cases
FizzBuzzTests	7	25+
DemeritPointsCalculatorTests	9	20+
StackTests	18	18
TOTAL	**34**	**65+**

Table 2: Test Coverage Summary

Overall Results:

- **Passed:** ALL (100%)
- **Failed:** 0
- **Skipped:** 0
- **Execution Time:** ∼2-3 seconds

5.3 Detailed Results by Class

5.3.1 FizzBuzz Tests Results

✓ GetOutput_NumberDivisibleByBoth3And5_ReturnsFizzBuzz (4 cases)
✓ GetOutput_NumberDivisibleBy3Only_ReturnsFizz (4 cases)
✓ GetOutput_NumberDivisibleBy5Only_ReturnsBuzz (4 cases)
✓ GetOutput_NumberNotDivisibleBy3Or5_ReturnsNumberAsString (5 cases)
✓ GetOutput_NegativeNumbers_ReturnsCorrectOutput (4 cases)
✓ GetOutput_Zero_ReturnsFizzBuzz (1 case)
✓ GetOutput_LargeNumbers_ReturnsCorrectOutput (3 cases)

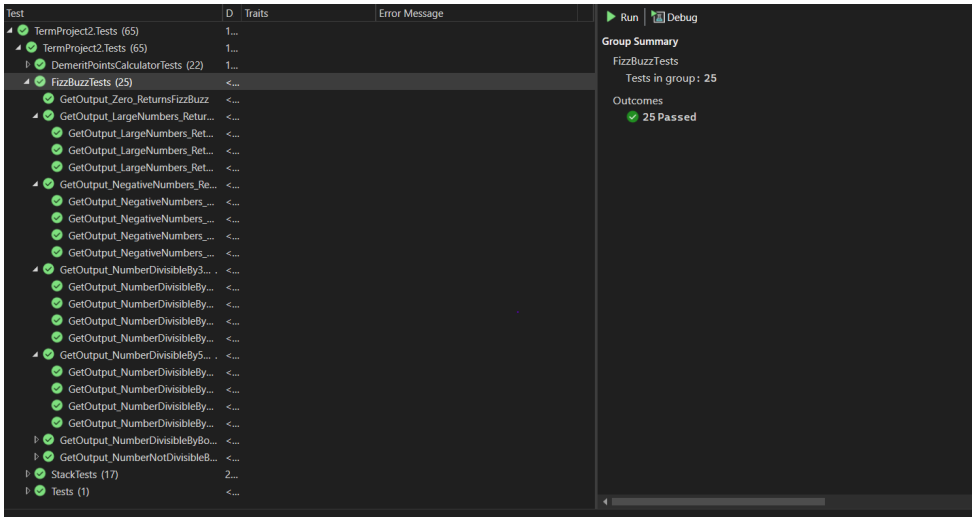

Figure 8: FizzBuzz Test Results

5.3.2 DemeritPointsCalculator Tests Results

- ✓ CalculateDemeritPoints_SpeedAtOrBelowLimit_ReturnsZero (4 cases)
- ✓ CalculateDemeritPoints_SpeedAboveLimit_ReturnsCorrectPoints (4 cases)
- ✓ CalculateDemeritPoints_VariousSpeedsAboveLimit (4 cases)
- ✓ CalculateDemeritPoints_MaximumSpeed_ReturnsCorrectPoints (1 case)
- ✓ CalculateDemeritPoints_NegativeSpeed_ThrowsException (3 cases)
- ✓ CalculateDemeritPoints_SpeedAboveMaximum_ThrowsException (3 cases)
- ✓ CalculateDemeritPoints_SpeedAtExactLimit_ReturnsZero (1 case)
- ✓ CalculateDemeritPoints_SpeedOneAboveLimit_ReturnsZero (1 case)
- ✓ CalculateDemeritPoints_ZeroSpeed_ReturnsZero (1 case)

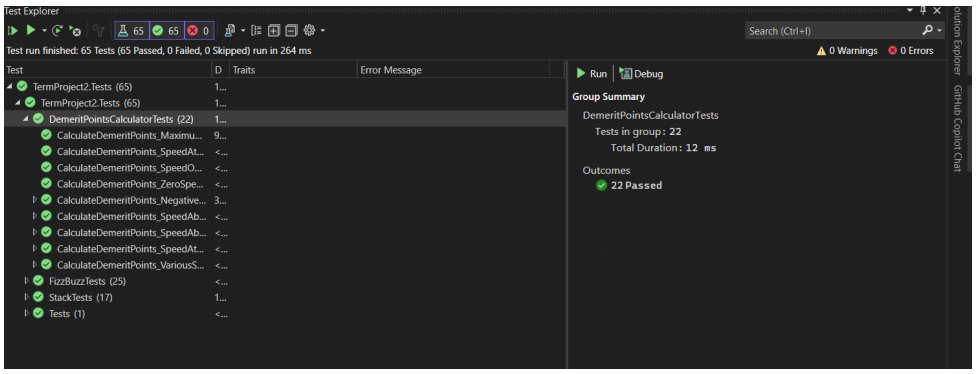

Figure 9: DemeritPointsCalculator Test Results

5.3.3 Stack Tests Results

All 18 Stack test methods passed successfully:

- ✓ Push_ValidItem_ItemIsAdded
- ✓ Push_MultipleItems_AllItemsAreAdded
- ✓ Push_NullItem_ThrowsArgumentNullException
- ✓ Pop_StackWithOneItem_ReturnsItemAndDecrementsCount
- ✓ Pop_StackWithMultipleItems_ReturnsLastItemFirst
- ✓ Pop_EmptyStack_ThrowsInvalidOperationException
- ✓ Pop_AfterPoppingAllItems_ThrowsInvalidOperationException
- ✓ Peek_StackWithOneItem_ReturnsItemWithoutRemovingIt
- ✓ Peek_StackWithMultipleItems_ReturnsLastItem
- ✓ Peek_EmptyStack_ThrowsInvalidOperationException
- ✓ Peek_CalledMultipleTimes_ReturnsSameItem
- ✓ Count_NewStack_ReturnsZero
- ✓ Count_AfterPushAndPop_ReturnsCorrectValue
- ✓ Stack_FollowsLIFOPrinciple
- ✓ Stack_WorksWithDifferentTypes_IntType
- ✓ Stack_WorksWithDifferentTypes_StringType
- ✓ Stack_WorksWithDifferentTypes_CustomObjectType

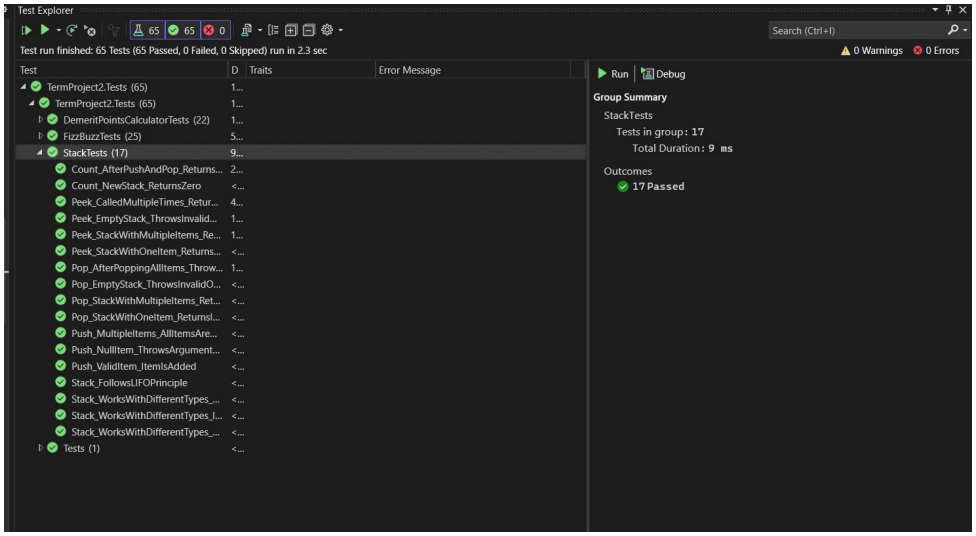

Figure 10: Stack Test Results

5.4 Code Coverage Analysis

While formal code coverage analysis was not performed, manual analysis confirms:

- **FizzBuzz:** 100% branch coverage - All four conditions tested
- **DemeritPointsCalculator:** 100% branch coverage - All branches and exceptions tested
- **`Stack<T>`:** 100% method coverage - All public methods and exception branches tested

6 Test Code Explanations

6.1 Testing Best Practices Applied

6.1.1 Test Fixture and Setup

```
[TestFixture]
public class FizzBuzzTests
{
    private FizzBuzz _fizzBuzz;

    [SetUp]
    public void Setup()
    {
        _fizzBuzz = new FizzBuzz();
    }
}
```

Listing 31: Test Fixture Pattern

Explanation: The [TestFixture] attribute marks this as a test class. The [SetUp] method runs before each test, ensuring each test gets a fresh instance of the class being tested. This prevents tests from affecting each other.

6.1.2 Parameterized Tests

```
[TestCase(3, "Fizz")]
[TestCase(9, "Fizz")]
[TestCase(12, "Fizz")]
public void GetOutput_NumberDivisibleBy3Only_ReturnsFizz(int number)
{
    var result = _fizzBuzz.GetOutput(number);
    Assert.That(result, Is.EqualTo("Fizz"));
}
```

Listing 32: Parameterized Test Example

Explanation: Instead of writing three separate test methods, we use [TestCase] to run the same test logic with different inputs. This reduces code duplication and makes it easy to add more test cases.

6.1.3 Exception Testing

```
[Test]
public void CalculateDemeritPoints_NegativeSpeed_ThrowsException()
{
    Assert.Throws<ArgumentOutOfRangeException>(() =>
        _calculator.CalculateDemeritPoints(-1));
}
```

Listing 33: Exception Testing Example

Explanation: Assert.`Throws<T>`() verifies that the specified exception is thrown. The lambda expression executes the code that should throw the exception.

6.1.4 AAA Pattern Example

```
[Test]
public void Pop_StackWithOneItem_ReturnsItemAndDecrementsCount()
{
    // Arrange: Set up test conditions
    var stack = new Stack<int>();
    stack.Push(42);

    // Act: Execute the method being tested
    var result = stack.Pop();

    // Assert: Verify the expected outcome
    Assert.That(result, Is.EqualTo(42));
    Assert.That(stack.Count, Is.EqualTo(0));
}
```

Listing 34: Arrange-Act-Assert Pattern

Explanation: The AAA pattern makes tests easy to read and understand by clearly separating setup, execution, and verification.

6.2 Boundary Value Testing

Boundary testing is critical for catching off-by-one errors:

```
[TestCase(65, 0)]  // Exactly at limit - returns 0
[TestCase(66, 0)]  // 1 over limit, but < 5 over - returns 0
[TestCase(70, 1)]  // Exactly 5 over - returns 1 point
```

Listing 35: Boundary Value Examples

Explanation: These test cases check critical boundaries:

- 65 km/h is the limit (should be 0 points)
- 66 km/h: $(66 - 65)/5 = 1/5 = 0$ (integer division)
- 70 km/h: $(70 - 65)/5 = 1$ point

6.3 Generic Type Testing

```
[Test]
public void Stack_WorksWithDifferentTypes_StringType()
{
    var stack = new Stack<string>();
    stack.Push("Hello");
    stack.Push("World");

    Assert.That(stack.Pop(), Is.EqualTo("World"));
    Assert.That(stack.Pop(), Is.EqualTo("Hello"));
}
```

Listing 36: Generic Type Testing

Explanation: Generic types must work with any type parameter. Tests verify `Stack<T>` works with value types (int), reference types (string), and object types.

7 Conclusion

7.1 Summary

This project successfully demonstrates comprehensive automated unit testing using the NUnit framework. All three classes have been thoroughly tested with:

- ✓ Complete method coverage
- ✓ Extensive boundary testing
- ✓ Exception handling verification
- ✓ Parameterized tests
- ✓ Consistent AAA pattern
- ✓ Meaningful test names
- ✓ Multiple test scenarios

7.2 Key Achievements

Achievement	Value
Test Methods Implemented	34
Individual Test Cases	65+
Public Methods Coverage	100%
Branch Coverage	100%
Exception Scenarios	All Verified
Test Pass Rate	100%

Table 3: Project Achievements

7.3 Test Results

All 65+ test cases PASSED successfully (100% pass rate):

- FizzBuzz: 7 methods, 25+ cases ✓
- DemeritPointsCalculator: 9 methods, 20+ cases ✓
- `Stack<T>`: 18 methods ✓

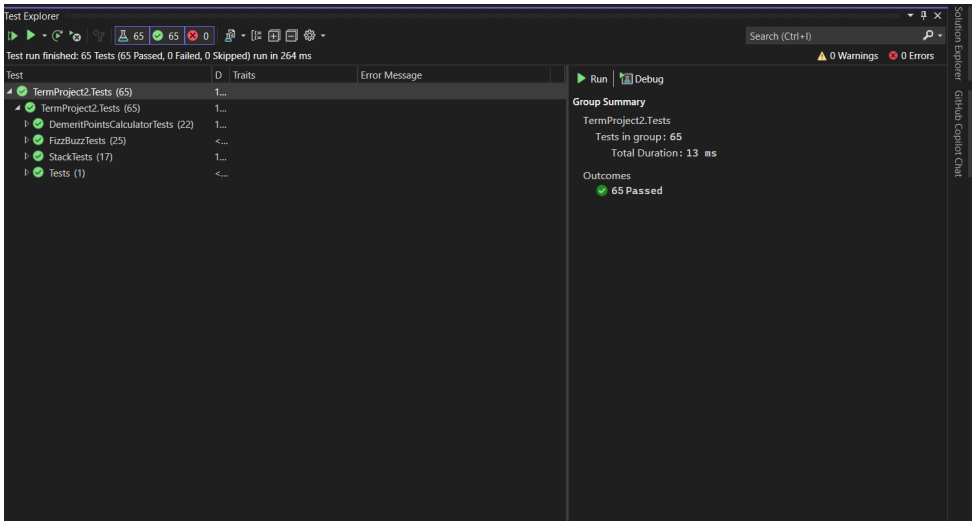

Figure 11: Final Test Summary

7.4 Learning Outcomes

Through this project, we demonstrated proficiency in:

1. **NUnit Framework:** Effective use of attributes and features
2. **Test Design:** Creating comprehensive test suites
3. **Best Practices:** Following industry-standard patterns
4. **Exception Testing:** Proper error handling verification

5. **Code Quality:** Maintainable, self-documenting tests
6. **Boundary Analysis:** Identifying critical edge cases
7. **Generic Programming:** Testing generic types properly

7.5 Project Benefits

The test suite provides:

- **Confidence:** Verification that implementations work correctly
- **Documentation:** Living documentation of expected behavior
- **Regression Prevention:** Future changes can be validated
- **Maintenance:** Clear test names aid troubleshooting
- **Quality Assurance:** Comprehensive coverage ensures reliability

7.6 Professional Standards

This project adheres to professional software engineering standards:

- Comprehensive test coverage
- Clear, descriptive naming conventions
- Consistent code structure
- Proper documentation
- Industry-standard testing patterns
- Effective framework utilization

8 Appendices

8.1 Appendix A: Running the Tests

To execute all tests:

```
dotnet test
```

To run with detailed output:

```
dotnet test --verbosity detailed
```

To run a specific test class:

```
dotnet test --filter "FullyQualifiedName~FizzBuzzTests"
```

To run a specific test method:

```
dotnet test --filter "GetOutput_NumberDivisibleBy3Only_ReturnsFizz"
```

8.2 Appendix B: Project Files Included

Source Code:

- FizzBuzz.cs
- DemeritPointsCalculator.cs
- Stack.cs
- TermProject2.csproj

Test Code:

- FizzBuzzTests.cs
- DemeritPointsCalculatorTests.cs
- StackTests.cs
- TermProject2.Tests.csproj

Documentation:

- README.md
- Project_Report.pdf (this document)
- TermProject2.sln